

TEORÍA DE ALGORITMOS
(TB024) CURSO ECHEVARRÍA

Trabajo Práctico 1

19 de junio de 2026

Integrantes	Padrón
Joaquín Hernández	110470
Juan Francisco Cuevas	107963
Alexander Terrussi	107212
Dante Finzi	108456
Víctor Zacarías	107080

Índice

1. Ejercicio #2: Backup de antenas	3
1.1. Introduccion	3
1.2. Ejemplos	3
1.2.1. Caso con solucion $b = 2$	3
1.2.2. Caso sin solución: $b = 1$	3
1.3. Modelado del problema	4
1.4. Diseño de la solución	4
1.4.1. Adaptación al problema de flujo máximo	4
1.4.2. Pseudocódigo	5
1.5. Complejidad	6
1.6. Supuestos	7
1.7. Evaluaciones y resultados	7
1.8. Tiempos de ejecución	8
1.9. Seguimiento	10
1.9.1. Construcción de la red de flujo	10
1.9.2. Iteraciones de Edmonds-Karp	10
1.9.3. Verificación y reconstrucción	12

1. Ejercicio #2: Backup de antenas

1.1. Introduccion

El problema es el siguiente: se tiene una red de n antenas, todas conectadas entre si, con una matriz $d = d[1..n, 1..n]$ con las distancias entre todas las antenas. Para ellas se quiere asignar un backup a cada antena, de tal forma que cada antena tenga k vecinos capaces de hacerle backup pero con un límite b de estas. El objetivo es encontrar una asignación de backups que cumpla con estas restricciones, o determinar que no existe tal asignación.

1.2. Ejemplos

1.2.1. Caso con solución $b = 2$

Supongamos que tenemos 3 antenas en línea recta, con $k = 1$, $b = 2$ y $D = 10$. La matriz de distancias es la siguiente:

$$d = \begin{bmatrix} 0 & 8 & 16 \\ 8 & 0 & 8 \\ 16 & 8 & 0 \end{bmatrix}$$

Proponemos nombrar ‘vecinos’ a las antenas que se encuentran a una distancia menor a D .

Antena	Vecinos disponibles	Backup requerido	Opciones
1	{2}	$k = 1$	Antena 2
2	{1, 3}	$k = 1$	Antena 1 o 3
3	{2}	$k = 1$	Antena 2

Cuadro 1: Vecinos disponibles para cada antena (distancia $< D = 10$). Las antenas 1 y 3 tienen una única opción: elegir antena 2.

El problema surge inmediatamente: las antenas 1 y 3 no tienen opción — ambas *deben* elegir a la antena 2 como backup. Esto significa que la antena 2 aparecería en 2 conjuntos de backup (el de 1 y el de 3), llegando a la restricción $b = 2$. Luego, la antena 2 puede elegir entre sus vecinos {1, 3} para hacer backup. Sin embargo, si hubiéramos tenido $b = 1$, entonces la antena 2 no podría ser backup de ambas antenas 1 y 3, y no habría solución posible. Veamos las soluciones posibles:

Antena	Backup asignado	Apariciones como backup
<i>Opción A: Antena 2 elige antena 1</i>		
1	2	Aparece en: {2} → 1 vez
2	1	Aparece en: {1, 3} → 2 veces
3	2	Aparece en: {1, 3} → 2 veces
<i>Opción B: Antena 2 elige antena 3</i>		
1	2	Aparece en: {1, 3} → 2 veces
2	3	Aparece en: {2} → 1 vez
3	2	Aparece en: {1, 2} → 2 veces

Cuadro 2: Ambas opciones son válidas con $b = 2$: cada antena aparece en a lo sumo 2 conjuntos de backup.

1.2.2. Caso sin solución: $b = 1$

Consideremos el mismo ejemplo anterior pero con $b = 1$ en lugar de $b = 2$. Las antenas 1 y 3 *deben* elegir a la antena 2 como backup (es la única opción), lo que significa que la antena 2 aparecería en 2 conjuntos de backup. Esto **viola inmediatamente** la restricción $b = 1$ (máximo 1 aparición):

Antena	Intento de asignación	Apariciones
1	2 (obligatorio)	—
2	1 o 3	—
3	2 (obligatorio)	—
Análisis del conflicto		
La antena 2 aparecería como backup de las antenas 1 y 3, es decir, 2 apariciones. Pero $b = 1$ exige máximo 1 aparición. No importa lo que elija la antena 2 como su propio backup, ya ha violado la restricción.		

Cuadro 3: Con $b = 1$: la antena 2 debe ser backup de ambas 1 y 3 $\Rightarrow 2$ apariciones $> b = 1 \Rightarrow$ **No existe solución.**

1.3. Modelado del problema

Para resolver el problema, lo modelamos como un problema de flujo máximo en una red de flujo. La idea es construir una red de flujo tal que encontrar un flujo máximo en esa red corresponda a encontrar una asignación de backups que cumpla con las restricciones del problema.

Nuestra propuesta de modelado es la siguiente:

- Creamos un nodo fuente S y un nodo sumidero T .
- Para cada antena i , creamos dos nodos: A_i (representando la antena) y B_i (representando el backup de la antena).
- Conectamos el nodo fuente S a cada nodo A_i con una arista de capacidad k (el tamaño requerido del conjunto de backup).
- Conectamos cada nodo B_i al nodo sumidero T con una arista de capacidad b (el límite de apariciones como backup).
- Para cada par de antenas (i, j) que son vecinos (es decir, $d[i][j] < D$), conectamos el nodo A_i al nodo B_j con una arista de capacidad 1 (representando la posibilidad de que la antena i elija a la antena j como backup).
- El objetivo es encontrar un flujo máximo en esta red. Si el flujo máximo es igual a $n \cdot k$ (es decir, cada antena tiene asignados exactamente k backups), entonces existe una solución válida. De lo contrario, no existe tal asignación.

1.4. Diseño de la solución

1.4.1. Adaptación al problema de flujo máximo

La clave del diseño es reconocer que el problema tiene la estructura de una **asignación bipartita con cuotas**: de un lado están las antenas que *demandan* backups, del otro las antenas que *proveen* backups, y el flujo que pasa por la red representa exactamente qué antena respalda a cuál.

La red se construye con $2n + 2$ nodos organizados en cuatro capas:

1. **Fuente S** : inyecta k unidades de flujo hacia cada antena demandante. La capacidad k de la arista $S \rightarrow A_i$ fuerza a que cada antena obtenga *exactamente* k backups (ni más, ni menos, dado que el flujo total debe ser $n \cdot k$).
2. **Nodos A_i (demanda)**: cada antena i que necesita backup. El flujo que sale de A_i representa qué antenas conforman su conjunto de backup.
3. **Nodos B_j (oferta)**: cada antena j en su rol de proveedora de backup. La arista $A_i \rightarrow B_j$ existe solo si $d[i][j] < D$, con capacidad 1 (una antena puede aparecer a lo sumo una vez en el backup de cada otra).

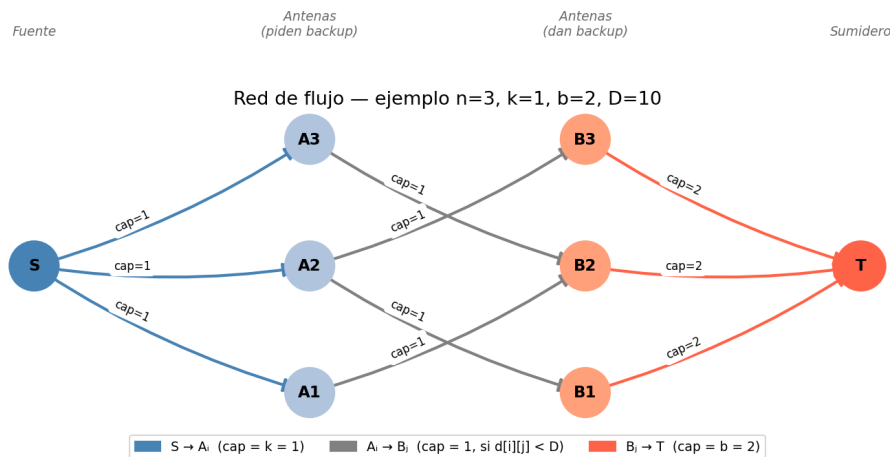


Figura 1: Red de flujo para el ejemplo con $n = 3, k = 1, b = 2, D = 10$. Los nodos A_i piden backup; los nodos B_j lo proveen. El flujo máximo $= n \cdot k = 3$ indica que existe solución. Generado con `grafico_red.py`.

4. **Sumidero T :** absorbe el flujo de todos los B_j . La capacidad b de la arista $B_j \rightarrow T$ limita cuántos conjuntos de backup distintos puede integrar la antena j .

Interpretación de la salida. Una vez ejecutado Edmonds-Karp, la respuesta se lee directamente de la red residual:

- Si **flujo máximo** $= n \cdot k$: existe solución. La arista $A_i \rightarrow B_j$ está saturada (capacidad residual $= 0$) si y solo si j pertenece al conjunto de backup de i .
- Si **flujo máximo** $< n \cdot k$: no existe solución. El corte mínimo certifica que algún subconjunto de antenas no puede obtener suficientes backups respetando el límite b .

1.4.2. Pseudocódigo

```

1  funcion OBTENER_CONJUNTO_BACKUP(distancias[1..n][1..n], k, b, D):
2
3      // --- Construccion de la red de flujo ---
4      S = 0
5      T = 2n + 1
6      cap[0..2n+1][0..2n+1] = 0          // matriz de capacidades residuales
7
8      para i = 1..n:
9          cap[S][i] = k                  // S -> A_i : antena i necesita k backups
10
11     para i = 1..n:
12         para j = 1..n:
13             si i != j y distancias[i][j] < D:
14                 cap[i][n+j] = 1        // A_i -> B_j : j puede ser backup de i
15
16     para j = 1..n:
17         cap[n+j][T] = b                // B_j -> T : j aparece en <= b conjuntos
18
19     // --- Flujo maximo (Edmonds-Karp) ---
20     flujo = EDMONDS_KARP(cap, S, T)
21
22     si flujo != n * k:
23         retornar SIN_SOLUCION          // corte minimo < n*k: imposible
24
25     // --- Reconstruccion de los conjuntos ---
26     conjuntos = lista vacia de n listas
    
```

```

27     para i = 1..n:
28         para j = 1..n:
29             si i != j y distancias[i][j] < D y cap[i][n+j] == 0:
30                 conjuntos[i].agregar(j) // arista saturada -> j es backup de i
31
32     retornar conjuntos
33
34
35 funcion EDMONDS_KARP(cap, S, T):
36     flujo_total = 0
37     repetir:
38         padre = BFS(cap, S, T) // camino minimo en aristas en red residual
39         si no existe camino S -> T:
40             terminar
41         // capacidad del cuello de botella
42         cuello = min{ cap[padre[v]][v] : v en camino S -> T }
43         // actualizar red residual
44         para cada arista (u, v) en el camino S -> T:
45             cap[u][v] -= cuello
46             cap[v][u] += cuello
47         flujo_total += cuello
48     retornar flujo_total

```

Listing 1: Pseudocódigo del algoritmo de backup de antenas

Estructuras de datos. La red residual se representa como una **matriz de adyacencia** $\text{cap}[u][v]$ de tamaño $(2n + 2) \times (2n + 2)$, donde $\text{cap}[u][v]$ almacena la capacidad residual de u a v . Esta elección simplifica la actualización de aristas inversas (siempre existe $\text{cap}[v][u]$) a costa de $O(n^2)$ de memoria, aceptable para los tamaños de instancia esperados.

1.5. Complejidad

El algoritmo tiene tres fases bien diferenciadas en el pseudocódigo. Designamos $V = 2n + 2$ la cantidad de nodos y E la cantidad de aristas de la red.

Fase 1 — Construcción de la red La inicialización de la matriz cap requiere $O(n^2)$ tiempo y espacio (la matriz tiene $(2n + 2)^2$ entradas). El lazo doble de las líneas 11–14 recorre todos los pares (i, j) con $i \neq j$ para verificar $d[i][j] < D$:

$$\underbrace{O(n)}_{S \rightarrow A_i} + \underbrace{O(n^2)}_{A_i \rightarrow B_j} + \underbrace{O(n)}_{B_j \rightarrow T} = O(n^2)$$

La cantidad de aristas que se agregan a la red es a lo sumo $E = n + n(n - 1) + n = O(n^2)$.

Fase 2 — Edmonds-Karp Cada iteración del ciclo principal ejecuta un BFS sobre la red residual. La red tiene $V = 2n + 2 = O(n)$ nodos y $E = O(n^2)$ aristas, por lo que cada BFS cuesta $O(V + E) = O(n^2)$.

Para acotar la cantidad de iteraciones usamos que todas las capacidades son enteras, por lo que cada camino aumentante incrementa el flujo en al menos 1. Como el flujo máximo es $n \cdot k$ (la demanda total de la red), el número de caminos aumentantes es a lo sumo $n \cdot k$. En consecuencia, la fase completa toma:

$$O(n \cdot k) \times O(n^2) = O(k \cdot n^3)$$

El límite clásico de Edmonds-Karp es $O(V \cdot E^2) = O(n^5)$, pero ese cota no aprovecha que el flujo máximo de *esta* red está acotado por $n \cdot k$ (usualmente mucho menor que el peor caso genérico).

Fase 3 — Reconstrucción El lazo doble recorre los mismos $O(n^2)$ pares (i, j) y lee directamente de la matriz residual si la arista está saturada. Costo: $O(n^2)$.

Costo total. La fase dominante es Edmonds-Karp:

$$O(k \cdot n^3)$$

Para los parámetros típicos del problema ($k \ll n$) esto es efectivamente $O(n^3)$, confirmando que el algoritmo es polinomial en el tamaño de la instancia.

1.6. Supuestos

Los supuestos sobre los que este algoritmo se basa son:

- Cuando el enunciado dice: "Plantear un algoritmo de complejidad polinomial que encuentre el conjunto de backup de tamaño k de cada una de las n antenas", se interpreta que el conjunto de backup de tamaño k se refiere a que cada antena debe tener por lo menos k antenas de backup asignadas, no que el grupo de backup de toda la red debe tener tamaño k .
- La red es conexa, es decir, existe un camino entre cualquier par de antenas.
- Las distancias entre antenas son simétricas, es decir, $d[i][j] = d[j][i]$ para todas las antenas i y j .
- Ninguna antena puede ser su propio backup.
- Las distancias entre antenas no son negativas.
- La matriz de distancias esta completa, incluyendo la diagonal principal (es decir, $d[i][i] = 0$ para todas las antenas i).
- Los parámetros k y b son enteros no negativos, y D es un número real positivo.
- No se considera el posicionamiento en el espacio de las antenas, solo sus distancias mutuas (ignoramos la representacion fisica de las antenas si estan en 2 o 3 dimensiones).
- El valor k es mucho menor a n , lo que hace que la fase de Edmonds-Karp sea efectivamente $O(n^3)$ en la práctica.
- El valor de b es menor o igual a n , ya que no tiene sentido que una antena pueda ser backup de más antenas que las que existen en la red.

1.7. Evaluaciones y resultados

Se ejecutaron 10 casos de prueba sobre el algoritmo implementado. Los casos fueron generados aleatoriamente con semilla fija (para reproducibilidad) mediante el script `generar_caso.py`, ubicando las antenas en coordenadas 2D uniformes dentro de un área de 100×100 , con distancias euclídeas. El caso 1 fue construido manualmente. Las mediciones se realizaron en una misma máquina y corresponden al tiempo de ejecución del algoritmo de flujo máximo (sin contar lectura de archivos).

Caso	Parametros	Resultado	Tiempo (ms)
caso1	n=5 k=2 b=3 D=7	Solucion	0.09
caso_b_igual_k	n=6 k=2 b=2 D=40.0	Sin solucion	0.08
caso_b_menor_k	n=8 k=3 b=2 D=40.0	Sin solucion	0.16
caso_d_chico	n=10 k=3 b=4 D=12.0	Sin solucion	0.09
caso_d_amplio	n=10 k=3 b=4 D=45.0	Sin solucion	0.35
caso_ajustado	n=15 k=2 b=3 D=35.0	Sin solucion	0.57
caso_matching_perfecto	n=20 k=1 b=1 D=30.0	Sin solucion	0.64
caso_mediano	n=30 k=4 b=5 D=50.0	Solucion	6.64
caso_grande	n=50 k=3 b=4 D=30.0	Solucion	19.31
caso_grande_imposible	n=50 k=3 b=4 D=15.0	Sin solucion	18.08

14

Listing 2: Resultados de ejecución sobre los 10 casos de prueba (generado por `ejecutar_casos.py`)

Observaciones.

- **caso_b_menor_k:** con $b = 2 < k = 3$, la capacidad total de la red ($n \cdot b = 16$) es menor a la demanda total ($n \cdot k = 24$), por lo que es imposible estructuralmente, independientemente de las distancias.
- **caso_d_chico vs. caso_d_amplio:** mismo layout geométrico (misma semilla), distinto D . Ambos resultan sin solución porque con $n = 10$ antenas dispersas en 100×100 , $D = 45$ no garantiza 3 vecinos por antena en esa distribución particular.
- **Tiempo de ejecución:** el tiempo crece con n , consistente con la complejidad $O(V \cdot E^2)$ de Edmonds-Karp. El caso **caso_grande_imposible** ($n = 50$) tarda casi lo mismo que **caso_grande** porque Edmonds-Karp igualmente explora la red antes de concluir que no hay solución.

1.8. Tiempos de ejecución

Para evaluar si los tiempos de ejecución se corresponden con la complejidad teórica $O(k \cdot n^3)$, se diseñó un experimento con parámetros fijos $k = 2$, $b = 3$, $D = 40$ y tamaños de entrada crecientes $n \in \{5, 10, 20, 30, 50, 75, 100, 150, 200\}$. Para cada valor de n se generaron 7 instancias aleatorias con semillas distintas, se descartaron el mínimo y el máximo, y se promedió el tiempo de las 5 mediciones restantes. Con k fijo, la complejidad predicha es $O(n^3)$; para contrastar, se ajustan simultáneamente una curva cúbica $f(n) = C_3 \cdot n^3$ y una cuadrática $f(n) = C_2 \cdot n^2$, ambas por mínimos cuadrados.

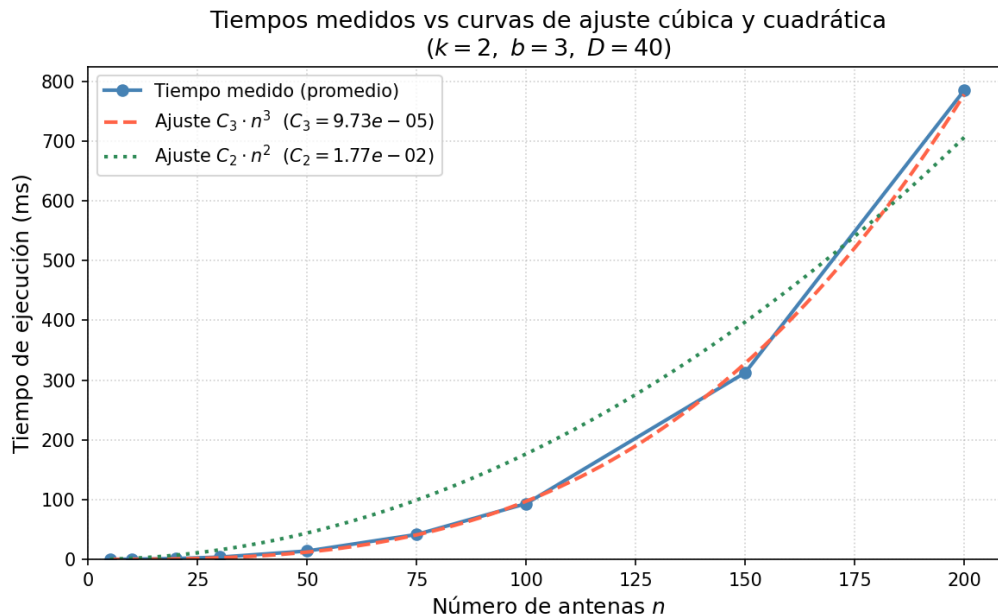


Figura 2: Tiempos de ejecución medidos (azul) frente al ajuste cúbico $C_3 \cdot n^3$ (rojo discontinuo) y el ajuste cuadrático $C_2 \cdot n^2$ (verde punteado), para $k = 2$, $b = 3$, $D = 40$. Generado con `analisis/tiempos.ipynb`.

Análisis del ajuste. La curva medida se ajusta con mayor precisión a $C_3 \cdot n^3$ que a $C_2 \cdot n^2$: para valores grandes de n la curva cuadrática subestima sistemáticamente los tiempos, mientras que la cúbica los sigue de cerca. Esto confirma la complejidad $O(k \cdot n^3)$ determinada teóricamente. La leve desviación para n pequeños se debe a que el costo constante de inicialización de la red domina sobre el término cúbico. En instancias sin solución, Edmonds-Karp recorre igualmente toda la red residual antes de concluir, por lo que el comportamiento asintótico es el mismo.

Efecto de los parámetros k , b y D . La Figura 3 muestra el tiempo en función de n al variar cada uno de los tres parámetros por separado.

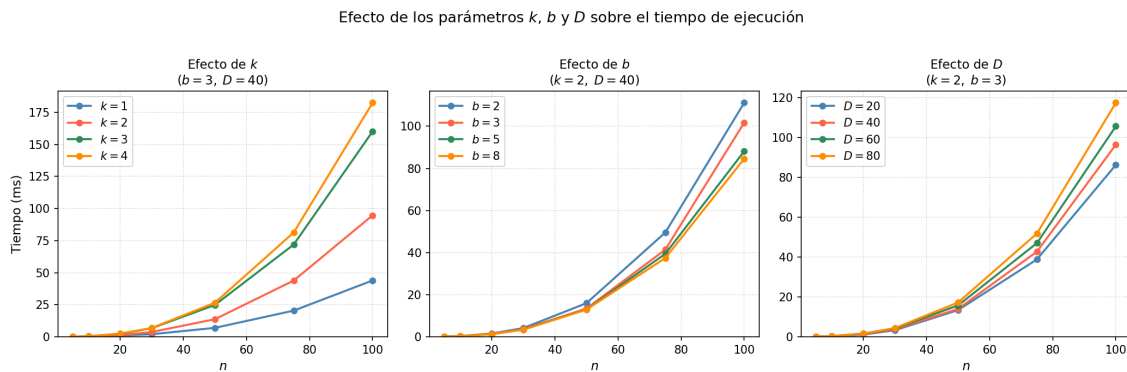


Figura 3: Efecto de k , b y D sobre los tiempos de ejecución. Cada panel varía un parámetro manteniendo los demás fijos. Generado con `analisis/tiempos.ipynb`.

- **Efecto de k :** al aumentar k , las curvas se desplazan hacia arriba de forma aproximadamente proporcional, consistente con la dependencia lineal $O(k \cdot n^3)$. Un mayor k implica más caminos aumentantes necesarios (el flujo objetivo es $n \cdot k$), por lo que Edmonds-Karp ejecuta más iteraciones.
- **Efecto de b :** valores mayores de b reducen el tiempo, ya que al relajar la restricción de capacidad en los nodos $B_j \rightarrow T$ el algoritmo encuentra caminos aumentantes con menos rerouting. Para b muy grande, la restricción efectiva pasa a ser k (cada A_i solo puede enviar k unidades), y el tiempo converge.
- **Efecto de D :** un umbral de distancia mayor aumenta la densidad de aristas intermedias $A_i \rightarrow B_j$, lo que incrementa el tamaño de la red y el costo de cada BFS. Esto se refleja en tiempos ligeramente mayores a medida que D crece, especialmente para n grande.

La Figura 4 superpone todas las curvas anteriores en un único gráfico, distinguiendo cada parámetro por estilo de línea y paleta de color.

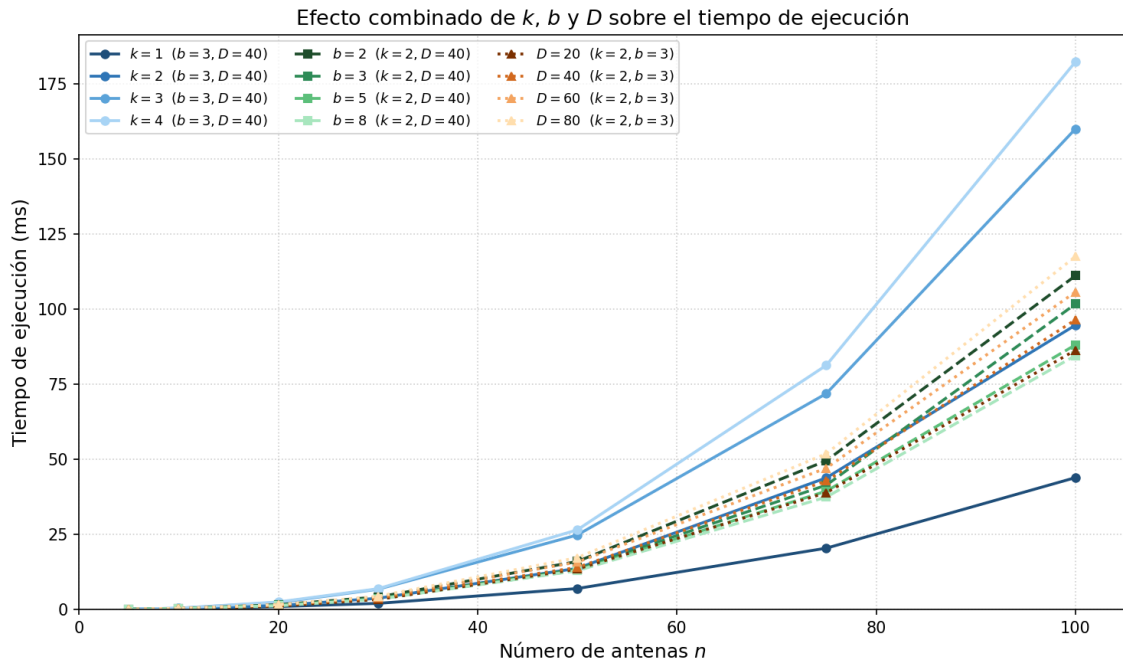


Figura 4: Efecto combinado de k (líneas sólidas, azules), b (líneas discontinuas, verdes) y D (líneas punteadas, naranjas) sobre el tiempo de ejecución en función de n . Es notorio el impacto de k en los tiempos de ejecución. Generado con `analisis/tiempos.ipynb`.

1.9. Seguimiento

Instancia. Seguimos la ejecución sobre `casos/caso1.toml`: $n = 5$ antenas con parámetros $k = 2$, $b = 3$, $D = 7$. El flujo objetivo es $n \cdot k = 5 \cdot 2 = 10$. La traza completa se puede reproducir con:

```
python antenas.py --verbose casos/caso1.toml
```

1.9.1. Construcción de la red de flujo

Con $D = 7$, se agrega la arista $A_i \rightarrow B_j$ ($\text{cap} = 1$) para cada par de antenas distintas con $d[i][j] < 7$. Las aristas $S \rightarrow A_i$ tienen capacidad $k = 2$ y las $B_j \rightarrow T$ tienen capacidad $b = 3$.

Antena i	Vecinos j con $d[i][j] < 7 \Rightarrow$ aristas $A_i \rightarrow B_j$
1	$j = 2$ ($d=5$), $j = 3$ ($d=6$)
2	$j = 1$ ($d=5$), $j = 3$ ($d=1$), $j = 4$ ($d=2$), $j = 5$ ($d=3$)
3	$j = 1$ ($d=3$), $j = 2$ ($d=1$), $j = 4$ ($d=1$), $j = 5$ ($d=2$)
4	$j = 2$ ($d=2$), $j = 3$ ($d=1$), $j = 5$ ($d=1$)
5	$j = 2$ ($d=6$), $j = 3$ ($d=2$), $j = 4$ ($d=1$)

Cuadro 4: Vecinos de cada antena y aristas intermedias generadas (todas con $\text{cap} = 1$).

La red resultante tiene $2n + 2 = 12$ nodos y $n + 20 + n = 30$ aristas (5 desde S , 20 intermedias, 5 hacia T).

1.9.2. Iteraciones de Edmonds-Karp

Cada iteración ejecuta un BFS sobre la red residual para hallar el camino más corto desde S hasta T , y aumenta el flujo por la capacidad del cuello de botella.

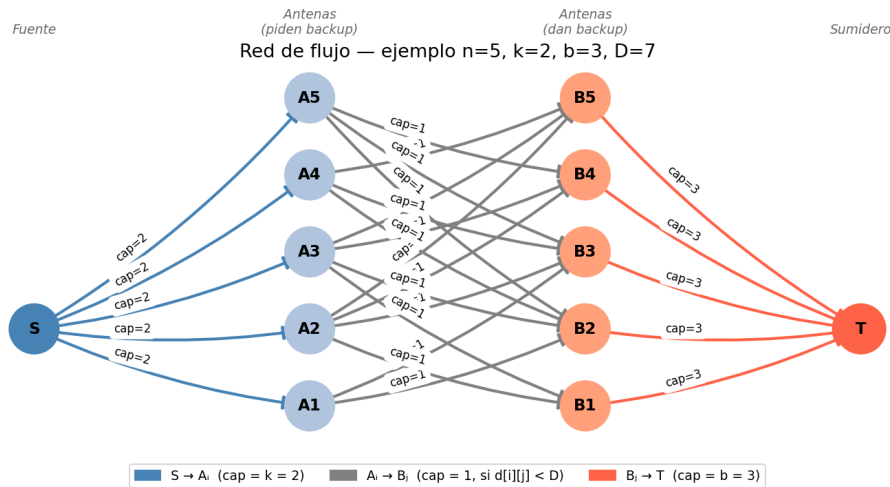


Figura 5: Red de flujo construida para el caso 1. Generado con `grafico_red.py`.

Iter.	Camino aumentante	Cuello	Flujo acum.
1	$S \rightarrow A_1 \rightarrow B_2 \rightarrow T$	1	1
2	$S \rightarrow A_1 \rightarrow B_3 \rightarrow T$	1	2
3	$S \rightarrow A_2 \rightarrow B_1 \rightarrow T$	1	3
4	$S \rightarrow A_2 \rightarrow B_3 \rightarrow T$	1	4
5	$S \rightarrow A_3 \rightarrow B_1 \rightarrow T$	1	5
6	$S \rightarrow A_3 \rightarrow B_2 \rightarrow T$	1	6
7	$S \rightarrow A_4 \rightarrow B_2 \rightarrow T$	1	7
8	$S \rightarrow A_4 \rightarrow B_3 \rightarrow T$	1	8
9	$S \rightarrow A_5 \rightarrow B_4 \rightarrow T$	1	9
10	$S \rightarrow A_5 \rightarrow B_2 \rightarrow A_3 \rightarrow B_4 \rightarrow T$ *	1	10

Cuadro 5: Caminos aumentantes encontrados por Edmonds-Karp. *La iteración 10 usa la arista residual $B_2 \rightarrow A_3$.

Nota sobre la iteración 10. Tras la iteración 9, A_5 aún necesita 1 unidad de flujo pero sus caminos directos están bloqueados: $B_2 \rightarrow T$ y $B_3 \rightarrow T$ están saturadas (cada una absorbió 3 unidades en las iteraciones 1,6,7 y 2,4,8 respectivamente), y $A_5 \rightarrow B_4$ fue usada en la iteración 9. El BFS encuentra entonces un camino de largo 5 que atraviesa la arista residual $B_2 \rightarrow A_3$ (el reverso de la arista $A_3 \rightarrow B_2$ saturada en la iteración 6). Esto cancela la asignación $3 \in \text{backup}(3)$ y la reemplaza por $4 \in \text{backup}(3)$, liberando a B_2 para que A_5 lo utilice. Este es el mecanismo central de Edmonds-Karp: la red residual permite “deshacer” decisiones anteriores en busca de un mejor flujo global.

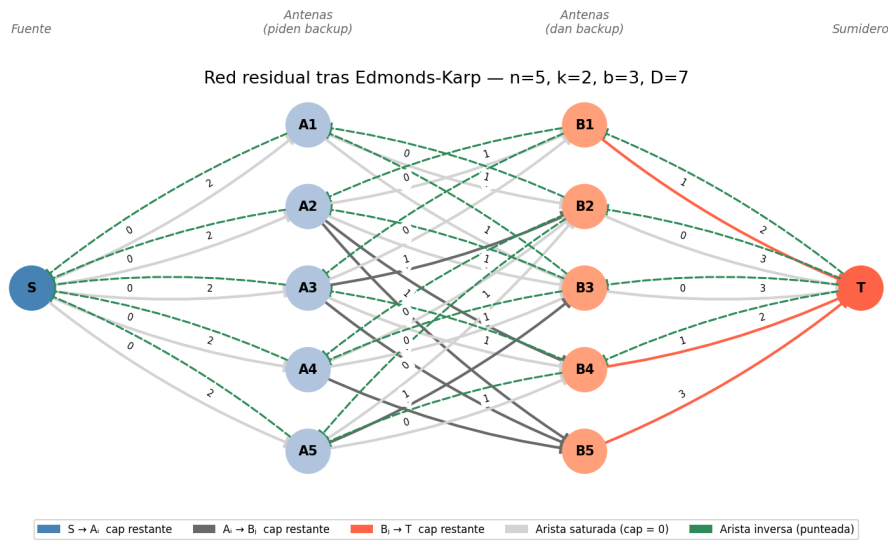


Figura 6: Red de flujo residual para el caso 1 señalando las aristas saturadas, las aristas con capacidad residual y los caminos. Generado con `grafico_red.py`.

1.9.3. Verificación y reconstrucción

El flujo total alcanzado es $10 = n \cdot k$, por lo que **existe solución**. Los conjuntos de backup se leen de las aristas saturadas (capacidad residual = 0) en la red final:

Antena i	Aristas saturadas $A_i \rightarrow B_j$	Backup	Apariciones como backup
1	$A_1 \rightarrow B_2, A_1 \rightarrow B_3$	$\{2, 3\}$	2 (antenas 4, 5)
2	$A_2 \rightarrow B_1, A_2 \rightarrow B_3$	$\{1, 3\}$	3 (antenas 1, 4, 5)
3	$A_3 \rightarrow B_1, A_3 \rightarrow B_4$	$\{1, 4\}$	3 (antenas 1, 2, 4)
4	$A_4 \rightarrow B_2, A_4 \rightarrow B_3$	$\{2, 3\}$	2 (antenas 3, 5)
5	$A_5 \rightarrow B_2, A_5 \rightarrow B_4$	$\{2, 4\}$	0

Cuadro 6: Reconstrucción final. Cada antena obtiene exactamente $k = 2$ backups y ninguna aparece en más de $b = 3$ conjuntos.

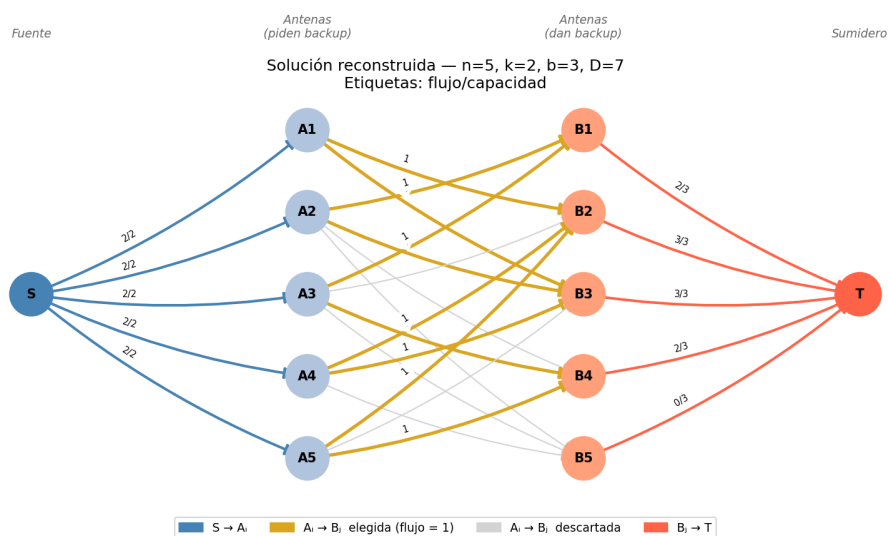


Figura 7: Aristas utilizadas en la reconstrucción de la solución. Generado con `grafico_red.py`.